

AI 游戏生成平台（Astrocade 类）技术架构文档

项目目标

构建一个：

用户通过一句自然语言，即可生成可运行、可分享、可迭代的游戏。

平台目标不是直接生成 AAA 游戏。

而是：

“AI 驱动的游戏工业化流水线”

核心价值：

- AI 自动生成游戏
- AI 自动拼装游戏
- AI 自动运行游戏
- AI 自动迭代游戏
- AI 自动形成 UGC 内容生态

最终形成：

用户一句话
→ AI游戏导演系统
→ 多Agent流水线
→ 自动生成游戏
→ 云端运行
→ 一键分享

一、平台核心理念

传统游戏工业：

策划 → 美术 → 程序 → 动画 → 音效 → 测试 → 发布

AI 游戏平台：

Prompt
→ AI导演
→ 多Agent协同

- 自动生成资产
- 自动生成玩法
- Runtime运行
- Feed传播

本质：

将游戏制作工业化、自动化、Agent化。

二、整体技术架构设计

平台采用：

“大脑 + 四肢”的多智能体分层架构

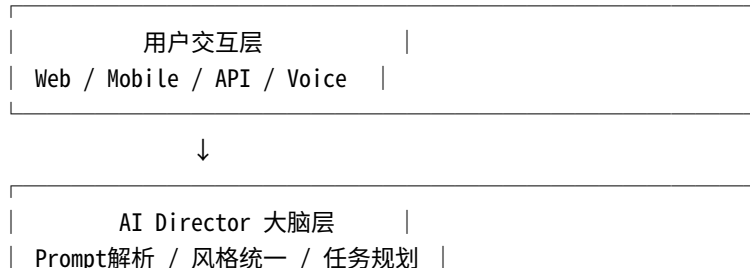
核心思想：

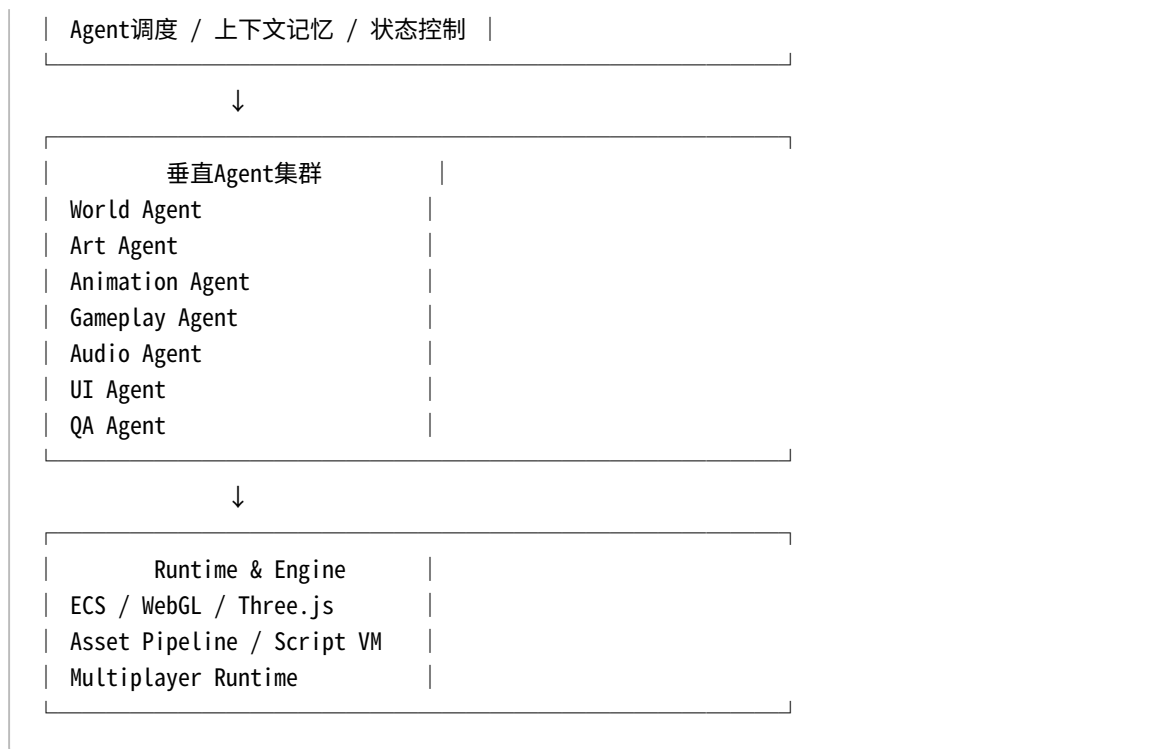
- AI Director 负责全局理解与调度
- 多Agent负责专业领域生成
- Runtime负责最终游戏运行
- MCP协议负责Agent通信

整体链路：

- 用户一句话
- 解析与概念生成
 - 任务分配与并行生成
 - 资产整合
 - 逻辑注入
 - 自动测试
 - 优化打磨
 - 发布分享

平台总体分层





核心 workflow

1. 用户输入一句话

例如：

做一个R0风格的冰雪副本
支持4人联机
Boss是冰龙

系统进入 AI Director。

2. 解析与概念生成

AI Director负责：

- Prompt理解
- 游戏类型识别
- 风格识别
- 玩法识别
- 生成GameSpec

输出：

GameSpec:
genre: RPG
theme: Snow
art_style: Anime
gameplay: Action
multiplayer: 4

3. 任务分配与并行生成

AI Director自动拆分任务。

例如：

World Agent
→ 地图生成

Art Agent
→ 场景与角色生成

Animation Agent
→ 动作生成

Gameplay Agent
→ 技能与Boss逻辑

Audio Agent
→ BGM与音效

所有Agent并行执行。

4. 资产整合

系统统一：

- Skeleton
- Shader
- 材质
- Scale
- Animation协议

最终生成：

统一游戏资产包

避免AI素材拼接感。

5. 逻辑注入

Gameplay Agent负责：

- 技能逻辑
- Boss机制
- AI行为树
- Buff系统
- 掉落逻辑

生成：

DSL → Runtime Script

自动注入Runtime。

6. 自动测试

QA Agent自动执行：

- NavMesh检测
- 卡死检测
- 性能检测
- 碰撞检测
- 平衡性检测

发现问题自动回流Agent修复。

7. 优化打磨

Director Agent负责：

- 镜头节奏
- 音乐节奏
- Shader统一
- 氛围统一
- 演出优化

最终形成统一体验。

8. 发布分享

系统自动：

- 云端部署
- 生成分享链接
- 生成Feed内容
- 支持Remix/Fork

最终形成UGC生态。

三、系统总架构

1. 用户层（User Layer）

用户输入：

做一个R0风格的雪山副本
支持4人联机
有冰龙Boss
战斗偏动作化

平台支持：

- Prompt生成
- 语音生成
- 图片生成
- 视频生成
- 游戏Remix

输出：

- Web Game
 - Mobile Game
 - Mini Game
 - Share Link
 - Feed内容
-

2. AI导演层（AI Director Layer）

这是平台最核心系统。

负责：

- 理解用户意图

- 拆解游戏结构
- 调度各Agent
- 控制风格统一
- 控制资产协议
- 控制Runtime生成

本质：

“AI游戏总导演”

AI Director 需要维护：

GameSpec:
Genre:
Theme:
ArtStyle:
Camera:
Combat:
Lighting:
MusicStyle:
Multiplayer:
RuntimeTarget:

3. 多Agent层（Agent Layer）

平台必须采用多Agent系统。

不能依赖单一模型。

3.1 世界观Agent

负责：

- 世界设定
- 地图结构
- 剧情背景
- 阵营设计
- 怪物生态

输出：

World:
Region:
Climate:

EnemyType:
Civilization:

3.2 美术Agent

负责：

- 场景生成
- 角色生成
- UI风格
- 材质风格
- Shader风格

要求：

必须统一风格。

不能出现：

- 写实 + 卡通混搭
- PBR + 二次元混搭

必须维护：

ArtBible:
Shader:
Material:
Rendering:
Outline:
Saturation:

3.3 动画Agent

负责：

- 动作生成
- 骨骼绑定
- 技能动画
- 表情动画
- 运镜动画

要求：

统一Skeleton协议。

例如：

Skeleton:
Humanoid_v2

否则动作无法复用。

3.4 Gameplay Agent

负责：

- 战斗逻辑
- 技能逻辑
- Buff系统
- AI行为树
- 掉落逻辑
- 副本机制

输出：

Skill:
Name:
Damage:
Range:
Cooldown:

最终转化：

- ECS
 - Lua
 - Behavior Tree
 - Runtime Script
-

3.5 音乐Agent

负责：

- BGM生成
- 环境音
- 战斗音效
- UI音效

重点：

音乐必须动态变化。

例如：

- Boss阶段变化
- 血量变化
- 战斗节奏变化

3.6 导演Agent（重点）

负责：

- 镜头节奏
- 情绪控制
- 演出控制
- 音乐切换
- Runtime氛围

例如：

玩家低血量：

- 镜头拉近
- 音乐低沉
- 屏幕泛红
- 呼吸增强

这是高级感来源。

3.7 QA Agent

负责：

- 自动测试
- Bug检测
- 卡死检测
- NavMesh检测
- 平衡性检测

目标：

实现 AI 自动修Bug。

四、资产协议层（Asset Protocol Layer）

这是平台最重要的工业层。

如果没有统一协议：

AI生成资产将无法拼装。

统一资产协议

所有AI生成资产必须符合统一规范。

例如：

Character:
Skeleton: Humanoid_v2
Shader: Anime_PBR
Rig: Standard
AnimationSet: Sword_Fast

场景协议：

Environment:
Lighting: Stylized
Scale: 1m
Collision: Enabled

音频协议：

Audio:
Format: OGG
Spatial: Enabled

五、Runtime层（核心）

平台必须有自己的 Runtime。

不能直接依赖完整 UE5 工业流程。

原因：

- 太重
- 无法秒级生成
- 无法实时拼装

推荐：

轻量Runtime

支持：

- ECS
- WebGL
- GPU Instancing
- 简化光照
- 自动LOD
- Streaming

平台目标：

秒级生成可运行游戏。

六、AI代码生成系统

AI游戏最难的不是美术。

而是：

Gameplay Logic

例如：

- Boss机制
- 技能CD
- Buff逻辑
- AI寻路
- 状态机

平台必须支持：

```
LLM
→ DSL
→ Runtime Script
```

例如：

```
Boss:
Phase1:
Skill: IceBreath
```

Phase2:
Skill: Blizzard

自动转：

- Lua
- ECS Component
- Behavior Tree

七、AI实时迭代系统

这是未来最大壁垒。

传统游戏：

改技能 → 重新开发 → 打包

AI Runtime：

“把火球改得更像龙焰”
→ 自动修改：

- 特效
- 动作
- 音效
- Shader
- 镜头

这叫：

Runtime AI Iteration

八、Feed与UGC系统

平台本质：

不是游戏引擎。

而是：

“游戏内容平台”

类似：

- TikTok
- Roblox
- YouTube Shorts

平台必须支持：

- Remix
- Fork
- 一键改游戏
- 一键分享
- Feed推荐

核心逻辑：

AI降低创作门槛
→ UGC爆发
→ Feed传播
→ 内容生态形成

九、技术架构建议

后端

推荐：

- Golang
- Rust
- NodeJS
- Python AI Worker

AI Orchestrator

推荐：

- Temporal
- LangGraph
- OpenClaw
- AutoGen
- CrewAI

推荐：

自研 Agent Runtime

因为未来一定会演变成：

AI工业操作系统

存储层

推荐：

- PostgreSQL
 - Redis
 - S3对象存储
 - Milvus向量数据库
-

Runtime

推荐：

- Unity Runtime
- Bevy ECS
- Godot Runtime
- Three.js Web Runtime

避免：

直接依赖完整 UE5 Editor。

十、平台工程架构设计

1. 网关层（Gateway Layer）

负责：

- 用户鉴权
- Prompt接入
- WebSocket通信
- 实时生成状态同步
- 游戏会话管理

推荐：

- Nginx
- Envoy
- API Gateway

2. AI Orchestrator层

负责：

- Agent调度
- Workflow编排
- 上下文共享
- 任务状态机
- 失败恢复

推荐：

User Prompt
→ Director Agent
→ Task Planner
→ Multi-Agent Execution
→ Runtime Builder

推荐技术：

- Temporal
- LangGraph
- OpenClaw
- 自研Workflow Runtime

3. Agent Runtime层

每个Agent独立运行。

支持：

- 动态扩缩容
- GPU调度
- 多模型路由
- Prompt缓存
- Context Memory

推荐架构：

Agent Runtime

- └── World Agent
- └── Art Agent
- └── Animation Agent
- └── Gameplay Agent
- └── Audio Agent
- └── QA Agent

4. 模型层 (Model Layer)

平台必须支持多模型。

不能绑定单一大模型。

推荐：

LLM Models

- └── GPT
- └── Claude
- └── Qwen
- └── DeepSeek

Image Models

- └── Flux
- └── SDXL
- └── Midjourney API

3D Models

- └── Trellis
- └── Hunyuan3D
- └── Tripo

Audio Models

- └── MusicGen
- └── AudioCraft

必须支持：

- Model Router
- Fallback机制
- 多模型投票
- Prompt Adapter

十一、数据结构设计

1. GameSpec

平台核心数据结构。

```
GameSpec:  
  id:  
  genre:  
  art_style:  
  gameplay:  
  runtime:  
  assets:  
  agents:
```

2. Asset Metadata

```
Asset:  
  id:  
  type:  
  style:  
  skeleton:  
  shader:  
  version:
```

3. Runtime Scene

```
Scene:  
  terrain:  
  entities:  
  navmesh:  
  lighting:  
  triggers:
```

4. Gameplay DSL

```
Skill:  
  id:
```

damage:
cooldown:
effect:

十二、任务执行流

游戏生成完整链路

用户输入Prompt
→ Prompt Parser
→ Director Agent
→ Task Planner
→ Agent Workflow
→ Asset Generation
→ Gameplay Generation
→ Runtime Builder
→ QA Agent
→ Game Packaging
→ Publish

实时迭代链路

用户修改Prompt
→ Diff Parser
→ Runtime Patch
→ Asset Rebuild
→ Hot Reload
→ 实时更新游戏

十三、部署架构

推荐云架构

CDN
↓
Gateway
↓
AI Orchestrator Cluster
↓

Agent Runtime Cluster



GPU Worker Cluster



Asset Storage



Runtime Server

GPU Worker设计

GPU节点负责：

- 图像生成
- 视频生成
- 3D生成
- 动作生成
- 音频生成

推荐：

- Kubernetes
- RunPod
- Volcano Scheduler
- Ray Cluster

存储层设计

PostgreSQL

→ 用户与项目数据

Redis

→ Runtime缓存

S3

→ 游戏资产

Milvus

→ 向量检索

十四、性能优化设计

必须解决的问题

1. 秒级生成

解决方案：

- 模板化生成
 - 资产缓存
 - 预生成素材库
 - Prompt Diff Patch
-

2. 风格统一

解决方案：

- Art Bible
 - Style Embedding
 - Shader统一
 - Skeleton统一
-

3. Runtime性能

解决方案：

- ECS
 - GPU Instancing
 - 自动LOD
 - Streaming
 - 轻量Physics
-

4. Agent一致性

解决方案：

- Shared Memory
 - Global Context
 - Director Agent统一调度
-

十五、安全与权限系统

支持：

- UGC审核
- Prompt安全过滤
- 版权检测
- AI资产水印
- 玩家举报系统

推荐：

Prompt Moderation
→ Asset Detection
→ Runtime Scan
→ Publish Review

十六、最终目标

平台最终形态：

用户一句话
→ AI导演系统
→ AI工业流水线
→ 自动生成游戏
→ 自动部署
→ 自动分享
→ 自动形成UGC生态

最终实现：

“一句话生成游戏世界”